# Virtualization

## Evolution of Virtualization

| Period | Milestone |
|---|---|
| 1960s | IBM mainframes (CP-40, CP/CMS) introduced time-sharing virtual machines so one expensive mainframe could be shared. |
| 1990s | Client-server era: one application per physical server; many servers using only 10-15% of their capacity. |
| 1999 | **VMware** brings virtualization to the x86 PC / server platform. |
| 2003-2008 | Open-source **Xen** (2003), **KVM** in Linux (2007), Microsoft **Hyper-V** (2008); hardware support from Intel VT-x and AMD-V. |
| 2010s | Cloud computing, **software-defined** infrastructure, VDI; hyper-converged systems. |
| 2013+ | **Containers** become mainstream (Docker, 2013) and **Kubernetes** (2014) orchestrates them. |
| Today | Hybrid and multi-cloud, serverless, edge computing; virtualization is the foundation of cloud. |

## Why Virtualization

- **Server consolidation:** many virtual servers on a few physical hosts; saves hardware, space, power and cooling.
- **Better resource utilization:** CPU and RAM are shared and balanced.
- **Cost savings:** lower capital and operational cost.
- **Fast provisioning:** a new VM can be created in minutes from a template.
- **Isolation:** a problem in one VM does not affect others.
- **High availability and mobility:** live migration (vMotion / Live Migration) moves VMs between hosts without downtime; automatic restart on failure.
- **Backup, snapshots and disaster recovery:** a VM is just files, so it is easy to copy, replicate and restore.
- **Testing and lab environments:** create and delete test systems safely.
- **Legacy support:** run old applications on old operating systems on new hardware.
- **Scalability and flexibility.**

## Types of Virtualizations

| Type | Explanation | Example |
|---|---|---|
| **Server virtualization** | One physical server runs multiple virtual servers | Hyper-V, VMware ESXi, KVM |
| **Desktop virtualization (VDI)** | User desktops run in the data center and are accessed remotely | Citrix, VMware Horizon, Azure Virtual Desktop |
| **Network virtualization** | Virtual switches, routers, firewalls and VLANs created in software (SDN / NFV) | VMware NSX, Hyper-V virtual switch |
| **Storage virtualization** | Combines multiple physical storage devices into one logical pool | SAN virtualization, Storage Spaces, vSAN |
| **Application virtualization** | Application runs in an isolated layer without full installation | App-V, Citrix apps, RemoteApp |
| **OS-level virtualization (containers)** | Applications are isolated while sharing the host OS kernel | Docker, LXC |
| **Data virtualization** | Provides a single view of data from many sources without moving it | Data virtualization platforms |

## Hypervisors

A **hypervisor** (virtual machine monitor, VMM) is the software layer that creates and manages virtual machines and shares physical hardware among them.

| Point | Type 1 (bare-metal) | Type 2 (hosted) |
|---|---|---|
| Runs on | Directly on the physical hardware | On top of a normal operating system |
| Performance | High, efficient | Lower, because of the extra OS layer |
| Use | Production data centers | Labs, desktops, testing, learning |
| Examples | VMware ESXi, Microsoft Hyper-V, KVM, Xen, Proxmox | VMware Workstation, Oracle VirtualBox, VMware Fusion, Parallels |

> **Hyper-V:** Hyper-V is a Type 1 hypervisor. Even when you enable it on a Windows Server, the Windows host OS becomes a special "parent partition" running on top of the hypervisor.

# Hyper-Converged Infrastructure (HCI)

**HCI** combines **compute (servers), storage and networking** into one software-defined solution running on standard x86 servers (nodes). Instead of a separate SAN storage box and switches, the **local disks of every node** are combined into a shared storage pool by software, and a hypervisor plus management software controls everything.

## Main characteristics

- **Software-defined** storage, compute and networking.
- **Scale-out:** need more capacity? just add another node.
- **Single management console** for the whole stack.
- **Built-in data protection:** data is replicated or erasure coded across nodes, so a node failure does not lose data.
- **Simplified operations** and faster deployment.

# HCT vs Traditional Infrastructure

(In the TOC this is HCI vs traditional infrastructure.)

| Point | Traditional (three-tier) | Hyper-Converged |
|---|---|---|
| Components | Separate servers, SAN / NAS storage arrays and storage network switches | Standard servers with local disks forming a software-defined pool |
| Management | Different tools and specialists for compute, storage, network | Single unified management |
| Scaling | Scale each tier separately; often scale-up with big purchases | Scale-out by adding nodes in small steps |
| Deployment time | Weeks / months | Days / hours |
| Cost | High upfront (special hardware), flexible hardware choice | Lower upfront, predictable cost per node |
| Performance | Excellent for very large or specialized workloads | Good for most general workloads; network important |
| Complexity | High | Low |
| Best for | Very large, specialized or legacy systems | Branch offices, VDI, private cloud, general virtualization |

## Leading HCI Platforms

| Platform | Notes |
|---|---|
| **Nutanix** (AOS + AHV) | One of the pioneers; includes its own hypervisor (AHV); works with other hypervisors |
| **VMware vSAN / VMware Cloud Foundation** | Storage built into the ESXi hypervisor |
| **Microsoft Azure Stack HCI** (now Azure Local) | Windows Server / Hyper-V based, with Storage Spaces Direct, integrated with Azure |
| **Dell VxRail / HPE SimpliVity / Cisco HyperFlex** | Vendor appliances (some of these have changed or reached end of sale over time) |
| **Proxmox VE + Ceph / Red Hat OpenShift Virtualization** | Open-source or Linux-based options |

## Containers on HCI

HCI platforms are widely used as the base for **containers and Kubernetes**. A Kubernetes cluster can run in VMs on HCI or directly on the nodes, with HCI providing persistent storage for containers (through CSI drivers), networking and easy scaling. Many platforms (VMware Tanzu, Nutanix Karbon / NKP, Azure Kubernetes on Azure Local, Red Hat OpenShift) integrate Kubernetes so VMs and containers run side by side on the same infrastructure.

## Containerization

**Containerization** packages an application **with everything it needs** (code, libraries, dependencies, settings) into a lightweight, portable unit called a **container**. Containers share the host operating system's kernel, so they start in seconds and use far fewer resources than virtual machines.

> **Example:** The famous problem "it works on my laptop but not on the server" disappears, because the container carries the same environment everywhere (developer laptop, test, production, cloud).

### Benefits

- Lightweight and fast start-up (seconds).
- Portable across any system that runs a container engine.
- Consistent environments from development to production.
- Efficient resource use: higher density than VMs.
- Good fit for **microservices** and DevOps / CI-CD pipelines.
- Easy to scale up and down.

## VM vs Container

| Point | Virtual Machine | Container |
|---|---|---|
| Isolation level | Hardware-level; each VM has its own full guest OS | Process-level; shares the host OS kernel |
| Size | GBs (OS + app) | MBs (app + libraries) |
| Start time | Minutes | Seconds |
| Resource usage | Heavy (RAM and CPU for each OS) | Light |
| Security isolation | Stronger (separate kernels) | Weaker unless hardened (shared kernel) |
| Portability | Less portable (large images, hypervisor specific) | Highly portable |
| OS support | Different OS types on one host (Windows and Linux) | Containers share the host OS family (Linux containers on a Linux kernel) |
| Best for | Running full operating systems, legacy apps, strong isolation | Microservices, cloud-native apps, rapid scaling |

> **Remember:** They are **not enemies**. In practice containers often run **inside VMs** to combine VM security isolation with container agility.

## Container Ecosystem

| Component | Role |
|---|---|
| **Container image** | A read-only template containing the application and its dependencies |
| **Container** | A running instance of an image |
| **Container engine / runtime** | Docker Engine, containerd, CRI-O, Podman; builds and runs containers |
| **Dockerfile** | A text file with instructions to build an image |
| **Registry** | A place to store and share images (Docker Hub, Azure Container Registry, Amazon ECR, Harbor) |
| **Docker Compose** | Defines and runs multi-container applications on a single host |

| Component | Role |
| --- | --- |
| **Orchestrator** | Manages many containers across many hosts (Kubernetes, Docker Swarm) |
| **Networking and storage plugins** | Container networks (CNI) and persistent volumes (CSI) |
| **Security and monitoring tools** | Image scanning, runtime security, Prometheus / Grafana monitoring |

## Basic Docker commands

```
docker pull nginx            # download an image
docker run -d -p 80:80 nginx # start a container in the background, map port 80
docker ps                    # list running containers
docker images                # list local images
docker stop <container>      # stop a container
docker build -t myapp .      # build an image from a Dockerfile
```

# Kubernetes

**Kubernetes (K8s)** is an open-source **container orchestration platform** originally created by Google. When you have hundreds of containers across many servers, Kubernetes automates deploying, scaling, healing and updating them.

## Main architecture

| Component | Role |
| --- | --- |
| **Cluster** | The complete set of machines managed by Kubernetes |
| **Control plane (master)** | The brain: API server, scheduler, controller manager, etcd (the key-value database holding the cluster state) |
| **Worker node** | A machine that runs the application containers; runs the kubelet and a container runtime |
| **Pod** | The smallest unit; one or more containers sharing network and storage |
| **Deployment** | Declares how many copies (replicas) of a pod should run and handles updates |
| **Service** | Stable network address and load balancing for a set of pods |

| Component | Role |
|---|---|
| **Ingress** | Manages external HTTP/HTTPS access to services |
| **Namespace** | Logical separation inside a cluster |
| **ConfigMap / Secret** | Configuration data and sensitive data |
| **Persistent Volume (PV/PVC)** | Storage that survives container restarts |

### What Kubernetes gives you

- **Self-healing:** restarts failed containers and replaces unhealthy nodes.
- **Auto-scaling:** more pods when load increases.
- **Rolling updates and rollbacks** without downtime.
- **Service discovery and load balancing.**
- **Declarative configuration:** you describe the desired state (YAML) and Kubernetes makes it real.

```
kubectl get nodes

kubectl get pods

kubectl apply -f deployment.yaml

kubectl scale deployment myapp --replicas=5
```

## Virtualization, HCI & Containers in Cloud

Public clouds are built on exactly these technologies:

| Technology | Cloud examples |
|---|---|
| **Virtual machines** (IaaS) | Amazon EC2, Azure Virtual Machines, Google Compute Engine; the provider runs hypervisors (Nitro / KVM, Hyper-V) on huge clusters |
| **HCI / hybrid** | Azure Local (Azure Stack HCI), AWS Outposts, VMware Cloud on AWS, Nutanix Cloud Clusters: bring cloud-like infrastructure on-premises or run familiar platforms in the cloud |
| **Containers and Kubernetes (CaaS)** | Amazon EKS / ECS, Azure Kubernetes Service (AKS), Google GKE; the provider manages the control plane |
| **Serverless** | AWS Lambda, Azure Functions: code runs in lightweight containers or micro-VMs without managing servers |

**Journey:** physical servers → virtual machines → hyper-converged / private cloud → containers and Kubernetes → public cloud and serverless. Each step abstracts more of the hardware, making infrastructure more flexible and automated.